STAT 167 Linear Regression Implementation 1

Jaime Valencia Lopez

Contents

Section 0: EDA: Table Joins and Function Creation

Here, we will implement a linear regression model to predict the amount of delay a flight will have (arr_delay). We use a custom forward selection strategy to identify key predictors and evaluate the model using 10-fold cross-validation.

```r
# Load necessary libraries
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.1     v stringr   1.5.2
## v ggplot2   3.5.2     v tibble    3.2.1
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.1.0
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
library(nycflights13)
library(boot)
library(ggplot2)
library(dplyr)

# Set the random seed for reproducibility
set.seed(167)
```

Here we create the functions to calculate the MSE, as well as forward selection with lowest RSS.

```r
# Function to calculate Mean Squared Error (MSE)
calculateMSE <- function(y, yhat) {
  residuals <- y - yhat
  mse <- mean(residuals^2)
  return(mse)
}

# Automated Forward Selection Function
# This function picks the top 'n_vars' predictors based on the lowest RSS.
forward_selection <- function(data, response, n_vars) {
```

```r
  all_predictors <- names(data)[names(data) != response]
  selected_vars <- c()

  # Iterate one time in the outer loop for each variable
  for (i in 1:n_vars) {
    best_rss <- Inf
    best_var <- NULL
    remaining_vars <- setdiff(all_predictors, selected_vars)

    # Inner loop will iterate through all the variables
    for (var in remaining_vars) {
      # Choose the variable that has the lowest RSS value for the model
      formula_vars <- c(selected_vars, var)
      current_formula <- as.formula(paste(response, "~",
                                          paste(formula_vars, collapse = " + ")))

      # Fit model and calculate RSS
      fit <- lm(current_formula, data = data)
      current_rss <- sum(resid(fit)^2)

      # Update the best variable for this iteration
      if (current_rss < best_rss) {
        best_rss <- current_rss
        best_var <- var
      }
    }
    selected_vars <- c(selected_vars, best_var)
    cat("Step", i, ": Added", best_var, "(RSS:", round(best_rss, 2), ")\n")
  }
  return(selected_vars)
}
```

In this cell, we join all the tables to get the most comprehensive model possible.

```r
# Join all the tables and clean the data
flights_clean <- flights |>
  # Join the weather table by origin, year, month, day and hour
  inner_join(weather, by = c("origin", "year", "month", "day", "hour")) |>
  # Join the planes table by tailnum
  inner_join(planes, by = "tailnum", suffix = c("_flight", "_plane")) |>
  filter(dep_delay < 240) |>
  # Add carrier and origin to the select list
  select(dep_delay, hour, temp, visib, wind_speed, precip,
         seats, distance, origin, carrier, month) |>
  # Convert to factors so the lm() function treats them as categories
  mutate(
    origin = as.factor(origin),
    carrier = as.factor(carrier),
    month = as.factor(month)
  ) |>
  drop_na()
```

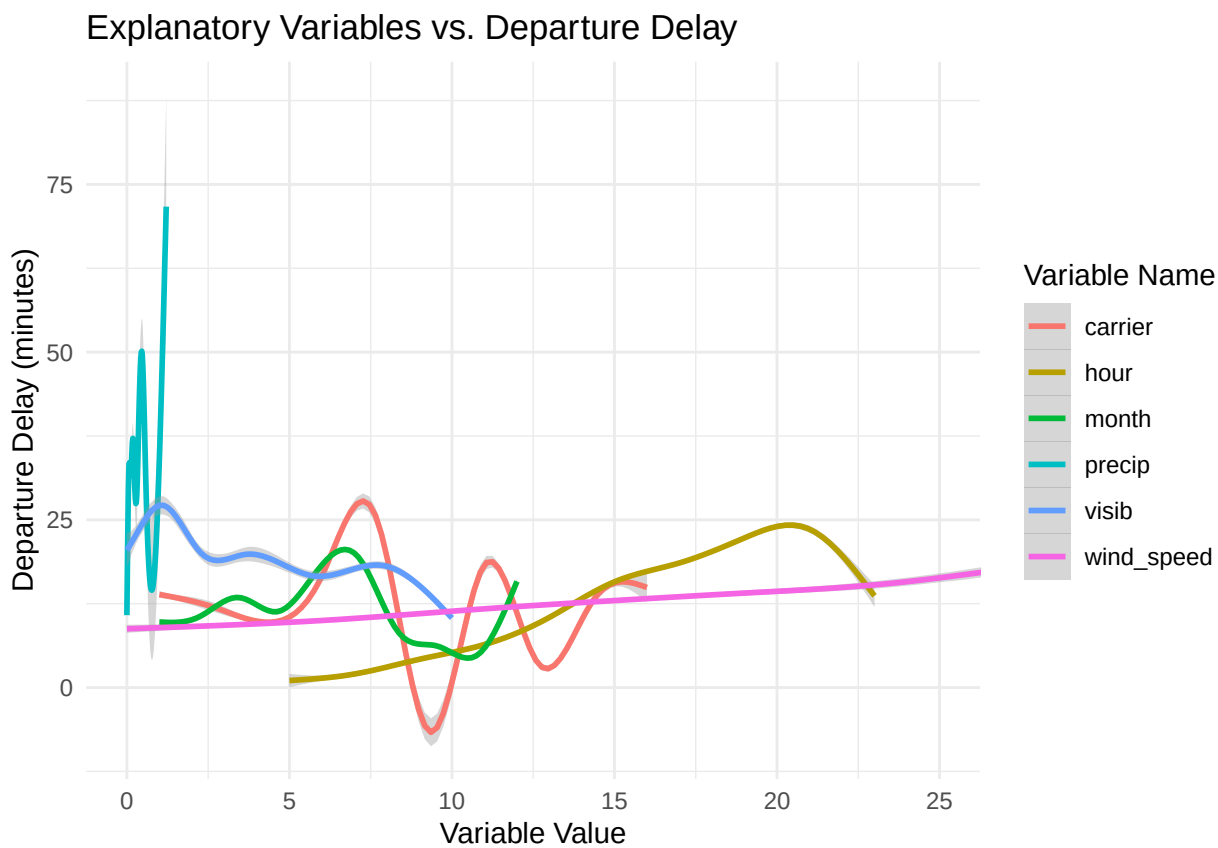
Retroactive EDA: Here, we visualize the relationship of the variables we will choose for our model in order to get a better understanding of their relationship with the response variable. Looking at each variable

individually, we see that their relationship is not inherently linear, which is why we will need make use of polynomial regression.

```r
# We "pivot" the explanatory variables into one column so they can be colored
plot_data <- flights_clean |>
  select(dep_delay, hour, month, carrier, visib, precip, wind_speed) |>
  # Convert everything to numeric for the scatter plot (except dep_delay)
  mutate(across(-dep_delay, as.numeric)) |>
  pivot_longer(
    cols = c(hour, month, carrier, visib, precip, wind_speed),
    names_to = "variable",
    values_to = "value"
  )

# 2. Create the single scatter plot
ggplot(plot_data, aes(x = value, y = dep_delay, color = variable)) +
  geom_smooth(alpha = 0.4) + # Alpha helps with overlapping points
  labs(
    title = "Explanatory Variables vs. Departure Delay",
    x = "Variable Value",
    y = "Departure Delay (minutes)",
    color = "Variable Name"
  ) +
  theme_minimal() +
  coord_cartesian(xlim = c(0,25))
```

```
## `geom_smooth()` using method = 'gam' and formula = 'y ~ s(x, bs = "cs")'
```



Section 1: Forward Selection Model with RSS

Here, we will call the forward selection model with the top 6 variables to predict departure delay. The adjusted R-squared value begins to plateu after adding more than 6 variables to the model.

```
# Run the forward selection to get the top 3 variables
top_vars <- forward_selection(data = flights_clean,
                              response = "dep_delay",
                              n_vars = 6)
```

```
## Step 1 : Added hour (RSS: 308140183 )
## Step 2 : Added month (RSS: 302440033 )
## Step 3 : Added carrier (RSS: 297681569 )
## Step 4 : Added visib (RSS: 294167505 )
## Step 5 : Added precip (RSS: 293519784 )
## Step 6 : Added wind_speed (RSS: 292968397 )
```

```
cat("\nTop selected variables:", paste(top_vars, collapse = ", "), "\n")
```

```
##
## Top selected variables: hour, month, carrier, visib, precip, wind_speed
```

Here, we can calculate the weighted 10-fold Cross Validation for our model.

```r
# Setup for 10-Fold CV
formula_cv <- as.formula(paste("dep_delay ~", paste(top_vars, collapse = " + ")))

# 2. Fit the model using glm()
glm.adv <- glm(formula_cv, data = flights_clean)

# 3. Run the 10-fold Cross-Validation
cv.result <- cv.glm(flights_clean, glm.adv, K = 10)

# 4. Extract raw CV estimate (MSE)
cat("cv.glm Weighted CV MSE:", round(cv.result$delta[1], 6), "\n")
```

```
## cv.glm Weighted CV MSE: 1057.256
```

Finally display the summary of our model.

```r
final_formula <- as.formula(paste("dep_delay ~", paste(top_vars, collapse = " + ")))
lm_final <- lm(final_formula, data = flights_clean)

summary(lm_final)
```

```
##
## Call:
## lm(formula = final_formula, data = flights_clean)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -84.397 -16.732  -7.794   2.783 244.199
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  0.96304    0.47967   2.008 0.044674 *
## hour         1.62627    0.01335 121.843  < 2e-16 ***
## month2       0.53475    0.31795   1.682 0.092598 .
## month3       2.97320    0.30572   9.725  < 2e-16 ***
## month4       4.18004    0.30602  13.659  < 2e-16 ***
## month5       2.31688    0.30368   7.629 2.37e-14 ***
## month6       9.55447    0.30771  31.051  < 2e-16 ***
## month7      11.49926    0.30506  37.695  < 2e-16 ***
## month8       4.09927    0.30514  13.434  < 2e-16 ***
## month9      -2.17128    0.30864  -7.035 2.00e-12 ***
## month10     -1.94575    0.30348  -6.411 1.44e-10 ***
## month11     -3.42053    0.30734 -11.130  < 2e-16 ***
## month12      6.12739    0.30933  19.808  < 2e-16 ***
## carrierAA   -4.08652    0.41069  -9.950  < 2e-16 ***
## carrierAS   -6.10396    1.24816  -4.890 1.01e-06 ***
## carrierB6   -2.30449    0.28549  -8.072 6.94e-16 ***
## carrierDL   -6.10198    0.28983 -21.053  < 2e-16 ***
## carrierEV    5.61520    0.28787  19.506  < 2e-16 ***
## carrierF9    1.31286    1.32498   0.991 0.321760
## carrierFL    0.79237    0.64721   1.224 0.220847
## carrierHA   -7.06609    1.78683  -3.955 7.67e-05 ***
```

```
## carrierMQ   -6.20488    1.08983  -5.693 1.25e-08 ***
## carrierOO   -2.95381    6.04398  -0.489 0.625041
## carrierUA   -1.53434    0.28433  -5.396 6.81e-08 ***
## carrierUS   -8.33809    0.34036 -24.498  < 2e-16 ***
## carrierVX   -1.92202    0.52073  -3.691 0.000223 ***
## carrierWN    3.77417    0.38864   9.711  < 2e-16 ***
## carrierYV    0.64867    1.42237   0.456 0.648356
## visib       -1.64073    0.03390 -48.405  < 2e-16 ***
## precip      51.66513    2.22904  23.178  < 2e-16 ***
## wind_speed   0.26632    0.01166  22.838  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 32.51 on 277138 degrees of freedom
## Multiple R-squared:  0.09701,    Adjusted R-squared:  0.09691
## F-statistic: 992.4 on 30 and 277138 DF,  p-value: < 2.2e-16
```

We have a pretty low Adjusted R-squared value of 0.09691, meaning that we can only explain a bit lower than 10% of the variance in our response variable. The MSE of 1057.256 means that there is an average error of around 32.5 minutes per prediction. We have established that the relationship between the explanatory variables and the response variable is not necessary linear, therefore, we will implement a model in which we use polynomial regression to augment our model prediction power.

Section 2: Polynomial Regression GAM Model

Here, we will use the 10-fold validation model, processing the variables that we have already selected with the best RSS values. We take a look at each variable and test their 10-fold CV MSE value from powers 1 through 5 and select the power that has the lowest MSE value.

```r
# List of numeric variables to test
vars_to_test <- c("hour", "visib", "wind_speed", "precip")
max_poly <- 5

# Create a data frame to store results for plotting
cv_results_df <- data.frame()

for (v in vars_to_test) {
  cat("Processing variable:", v, "\n")

  for (d in 1:max_poly) {
    # Dynamically create the formula: dep_delay ~ poly(var, d)
    formula_str <- as.formula(paste("dep_delay ~ poly(", v, ",", d, ")"))

    # Fit the GLM
    fit <- glm(formula_str, data = flights_clean)

    # Calculate 10-fold CV
    # Note: Use a seed for reproducibility if comparing small differences
    set.seed(123)
    cv_err <- cv.glm(flights_clean, fit, K = 10)$delta[1]

    # Save result
    cv_results_df <- rbind(cv_results_df, data.frame(Variable = v, Degree = d, MSE = cv_err))
  }
}
```

```
## Processing variable: hour
## Processing variable: visib
## Processing variable: wind_speed
## Processing variable: precip
```

```r
# Find the degree with the minimum MSE for each variable
optimal_values <- cv_results_df %>%
  group_by(Variable) %>%
  filter(MSE == min(MSE)) %>%
  select(Variable, Degree, MSE)

print(optimal_values)
```

```
## # A tibble: 4 x 3
## # Groups:   Variable [4]
##   Variable   Degree   MSE
##   <chr>       <int> <dbl>
## 1 hour            5 1110.
## 2 visib           4 1160.
## 3 wind_speed      5 1167.
## 4 precip          5 1155.
```

```r
glm.optimal <- glm(dep_delay ~ poly(hour, 5, raw = TRUE) +
                   month +
                   carrier +
                   poly(visib, 4, raw = TRUE) +
                   poly(precip, 5, raw = TRUE) +
                   poly(wind_speed, 5, raw = TRUE),
                   data = flights_clean)

# Perform 10-fold CV to get the MSE
cv.optimal <- cv.glm(flights_clean, glm.optimal, K = 10)

# Output the result
cat("Optimal Model 10-fold CV MSE:", cv.optimal$delta[1], "\n")
```

```
## Optimal Model 10-fold CV MSE: 1050.117
```

The optimal model 10-fold CV MSE is 1050.117 , meaning that there is a slight improvement from the previous linear model. It has decreased by around 7 points.

```r
lm.optimal <- lm(dep_delay ~ poly(hour, 5, raw = TRUE) +
                   month +
                   carrier +
                   poly(visib, 4, raw = TRUE) +
                   poly(precip, 5, raw = TRUE) +
                   poly(wind_speed, 5, raw = TRUE),
                   data = flights_clean)

summary(lm.optimal)
```

```
##
```

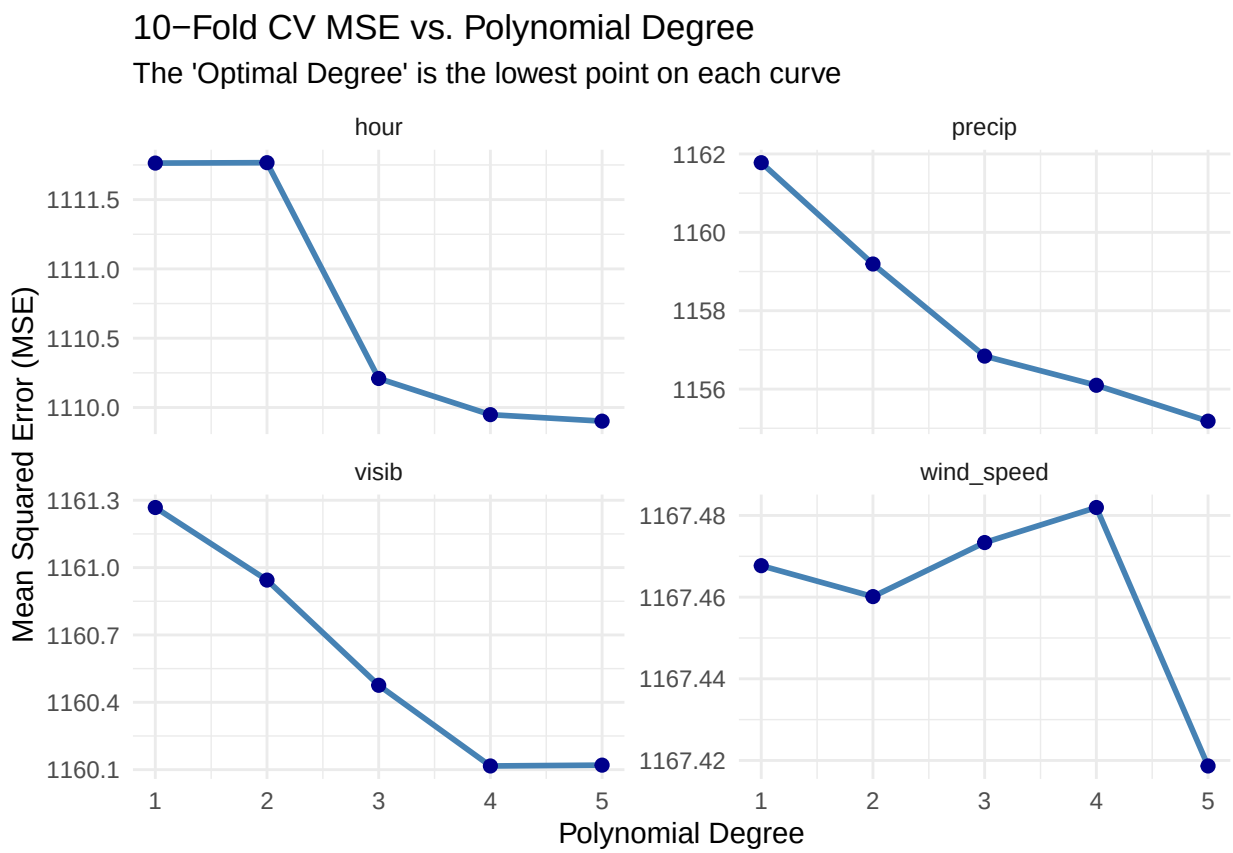
```
## Call:
## lm(formula = dep_delay ~ poly(hour, 5, raw = TRUE) + month +
##     carrier + poly(visib, 4, raw = TRUE) + poly(precip, 5, raw = TRUE) +
##     poly(wind_speed, 5, raw = TRUE), data = flights_clean)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -71.470 -16.750  -7.522   2.844 243.650
##
## Coefficients:
##                                  Estimate Std. Error t value Pr(>|t|)
## (Intercept)                     1.962e+01  1.117e+01   1.757 0.078920 .
## poly(hour, 5, raw = TRUE)1     -2.957e+00  4.805e+00  -0.615 0.538342
## poly(hour, 5, raw = TRUE)2      6.346e-01  7.895e-01   0.804 0.421535
## poly(hour, 5, raw = TRUE)3     -6.137e-02  6.179e-02  -0.993 0.320604
## poly(hour, 5, raw = TRUE)4      3.424e-03  2.315e-03   1.479 0.139177
## poly(hour, 5, raw = TRUE)5     -7.314e-05  3.340e-05  -2.190 0.028543 *
## month2                          1.393e-01  3.179e-01   0.438 0.661187
## month3                          2.774e+00  3.060e-01   9.063  < 2e-16 ***
## month4                          4.206e+00  3.059e-01  13.750  < 2e-16 ***
## month5                          2.290e+00  3.035e-01   7.546 4.50e-14 ***
## month6                          9.285e+00  3.084e-01  30.109  < 2e-16 ***
## month7                          1.144e+01  3.057e-01  37.407  < 2e-16 ***
## month8                          3.765e+00  3.052e-01  12.336  < 2e-16 ***
## month9                         -2.137e+00  3.082e-01  -6.934 4.09e-12 ***
## month10                        -1.852e+00  3.031e-01  -6.109 1.00e-09 ***
## month11                        -3.566e+00  3.075e-01 -11.596  < 2e-16 ***
## month12                         5.802e+00  3.097e-01  18.733  < 2e-16 ***
## carrierAA                      -3.670e+00  4.097e-01  -8.959  < 2e-16 ***
## carrierAS                      -6.956e+00  1.245e+00  -5.586 2.33e-08 ***
## carrierB6                      -1.453e+00  2.885e-01  -5.037 4.74e-07 ***
## carrierDL                      -5.877e+00  2.890e-01 -20.334  < 2e-16 ***
## carrierEV                       6.152e+00  2.884e-01  21.333  < 2e-16 ***
## carrierF9                       1.375e+00  1.321e+00   1.041 0.297875
## carrierFL                       1.238e+00  6.456e-01   1.918 0.055088 .
## carrierHA                      -5.362e+00  1.783e+00  -3.007 0.002640 **
## carrierMQ                      -5.733e+00  1.086e+00  -5.277 1.31e-07 ***
## carrierOO                      -3.285e+00  6.023e+00  -0.545 0.585446
## carrierUA                      -1.301e+00  2.845e-01  -4.573 4.80e-06 ***
## carrierUS                      -7.962e+00  3.409e-01 -23.359  < 2e-16 ***
## carrierVX                      -1.649e+00  5.192e-01  -3.176 0.001496 **
## carrierWN                       4.117e+00  3.884e-01  10.599  < 2e-16 ***
## carrierYV                       6.585e-01  1.418e+00   0.464 0.642440
## poly(visib, 4, raw = TRUE)1    -7.067e+00  1.020e+00  -6.931 4.19e-12 ***
## poly(visib, 4, raw = TRUE)2     7.489e-01  3.811e-01   1.965 0.049390 *
## poly(visib, 4, raw = TRUE)3     7.374e-02  5.289e-02   1.394 0.163242
## poly(visib, 4, raw = TRUE)4    -9.623e-03  2.426e-03  -3.967 7.29e-05 ***
## poly(precip, 5, raw = TRUE)1    3.343e+02  1.302e+01  25.667  < 2e-16 ***
## poly(precip, 5, raw = TRUE)2   -2.210e+03  1.360e+02 -16.244  < 2e-16 ***
## poly(precip, 5, raw = TRUE)3    5.714e+03  4.499e+02  12.702  < 2e-16 ***
## poly(precip, 5, raw = TRUE)4   -6.062e+03  5.523e+02 -10.975  < 2e-16 ***
## poly(precip, 5, raw = TRUE)5    2.218e+03  2.218e+02  10.000  < 2e-16 ***
## poly(wind_speed, 5, raw = TRUE)1 -5.241e-01  1.602e-01  -3.272 0.001068 **
## poly(wind_speed, 5, raw = TRUE)2  1.083e-01  3.004e-02   3.605 0.000312 ***
```

```
## poly(wind_speed, 5, raw = TRUE)3 -7.109e-03  2.305e-03  -3.084 0.002040 **
## poly(wind_speed, 5, raw = TRUE)4  2.187e-04  7.524e-05   2.907 0.003646 **
## poly(wind_speed, 5, raw = TRUE)5 -2.363e-06  8.648e-07  -2.733 0.006286 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 32.4 on 277123 degrees of freedom
## Multiple R-squared:  0.1033, Adjusted R-squared:  0.1031
## F-statistic: 709.3 on 45 and 277123 DF,  p-value: < 2.2e-16
```

Additionally, we see here that the adjusted R-Squared value has increased to 0.1031 from the previous 0.09691; we can now explain over 10% of the variance in our response variable.

```r
ggplot(cv_results_df, aes(x = Degree, y = MSE)) +
  geom_line(color = "steelblue", size = 1) +
  geom_point(color = "darkblue", size = 2) +
  facet_wrap(~Variable, scales = "free_y") +
  theme_minimal() +
  labs(title = "10-Fold CV MSE vs. Polynomial Degree",
       subtitle = "The 'Optimal Degree' is the lowest point on each curve",
       x = "Polynomial Degree",
       y = "Mean Squared Error (MSE)")
```

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```




```r
# Set up a 2x2 grid to see all plots at once
par(mfrow = c(2, 2))

# Generate diagnostic plots
dev.new(width=10, height=8)
plot(lm.optimal)

# Reset grid
par(mfrow = c(1, 1))
```

While we were able to achieve better results while using polynomial multiple regression, we still were only to explain around 10% of the variance with our model. We also note that the residuals indicate that our model was better at interpreting lower department delay numbers. While not perfect, this model can be used to predict general patterns of delay. More data and more complex models might be necessary to produce stronger predictions